CropWise: An Intelligent Digital Platform for Market Linkages and Price Discovery for Farmers

A Research and System Implementation Paper — Smart Markets. Better Prices. Stronger Farmers.

Problem Statement: PS-26132 — Strengthening Market Linkages and Price Discovery for Farmers

Track: Software | Theme: Agriculture, FoodTech & Rural Development

Smart India Hackathon (SIH) 2026 | Sponsoring Organisation: Government of Maharashtra (Maharashtra State Innovation Society)

Author(s): [Author Name(s) — To Be Filled]

Institution: [Institution / College Name — To Be Filled]

Department: [Department Name — To Be Filled]

Email: [Author Email — To Be Filled]

Repository: https://github.com/Abha153/cropwise

Live Deployment: https://cropwise-alpha.vercel.app/

Date: September 2026

Abstract

Smallholder and mid-sized farmers in India routinely make selling decisions — which crop to sell, in which market, to which buyer, and at what time — without reliable, decision-ready information. Government efforts such as the Agricultural Produce Market Committee (APMC) mandi system and the National Agriculture Market (eNAM) have improved auction transparency and published open price data, yet a persistent gap remains between raw price data and an actionable selling decision that accounts for transport cost, mandi fees, handling charges, buyer reliability, and produce quality. This gap, combined with fragmented buyer discovery, is formalised in Smart India Hackathon problem statement PS-26132, "Strengthening Market Linkages and Price Discovery for Farmers."

This paper presents CropWise, a full-stack web platform designed to close this gap for an individual farmer rather than a market as a whole. CropWise integrates market and arrival price intelligence, a transparent net-profit calculation that subtracts transport and handling costs from the sale price, a rule-based explainable recommendation engine (AgriAdvisor) for sell-now-versus-hold guidance, a farmer-facing marketplace (AgriMarket) with buyer verification and competing offers, buyer-matching ranked by estimated net profit and reliability, shared-transport pooling (FarmPool), cooperative group selling, and a multilingual (15 fully supported languages, 38 selectable) text and voice assistant. The system is implemented with a React 18 / Vite frontend, a Python FastAPI backend, SQLAlchemy over a SQLite database, and JWT-based authentication, and is deployed with a live frontend at cropwise-alpha.vercel.app.

The platform's decision-support and forecasting components are deliberately implemented as transparent, rule-based, and statistical models — a keyword-based multilingual intent engine, a linear-trend-and-volatility price forecaster, and a factor-scored recommendation engine — rather than as opaque machine-learning models, so that every output is accompanied by a plain-language explanation. The system runs end-to-end on realistic seeded demonstration data (ten Chhattisgarh markets, ten crops, sixty days of synthesised historical prices) with zero external dependencies, while also including a wired, testable integration path to two live data.gov.in government price resources that the system falls back from transparently when live data is unavailable, labelling every price record as either live or demo rather than blending the two.

This paper documents the problem context, reviews related digital-agriculture literature and existing platforms, and describes CropWise's system architecture, technology stack, functional modules, data model, and AI/decision-support methodology as they exist in the current implementation. Each feature is explicitly classified as implemented, partially implemented, seeded/fallback, externally dependent, or proposed future work, so that the paper's claims track the actual codebase rather than the platform's aspirational scope. The discussion is limited to what has been verified from the system's own documentation and design; no unverified performance, adoption, or accuracy figures are reported. The paper concludes with the system's current limitations and a realistic future scope for scaling the platform toward production-grade market linkage and price discovery support.

Keywords

Digital Agriculture; Market Linkage; Price Discovery; Farmer-Buyer Matching; Decision Support System; Agricultural Marketplace; Explainable Recommendation Engine; Multilingual Accessibility; eNAM; Mandi Price Intelligence.

Table of Contents

1. Introduction

Agriculture remains the primary livelihood for a substantial share of India's workforce, and the efficiency of agricultural output markets directly determines farm household income. Farmers do not merely need to produce a crop; they must also decide where to sell it, when to sell it, and to whom, under conditions of significant uncertainty. Unlike a manufacturer with contracted buyers, a smallholder farmer typically faces a perishable, time-sensitive harvest, several candidate markets at varying distances, and buyers of uneven reliability — all while having limited means to compare these options before committing to a sale.

Several structural challenges compound this uncertainty. First, price uncertainty: mandi (regulated wholesale market) prices vary from one market to the next and from one day to the next, and a farmer who has already transported produce to a market has little bargaining leverage to walk away. Second, lack of market transparency: even where price data is published, it is rarely presented in a way that supports a decision — a farmer must still mentally subtract transport cost, mandi fees, and handling charges, and weigh the risk of quality rejection, for every candidate market. Third, difficulty finding suitable buyers: traditional intermediaries (commission agents, local traders) often dominate price discovery and buyer access, and remote or first-time buyers are reluctant to bid on produce they cannot physically inspect. Fourth, information fragmentation: price data, buyer demand signals, transport economics, and quality assessment are typically available, if at all, from separate and disconnected sources. Fifth, logistics and storage constraints add further cost and risk to any selling decision, particularly for smallholders who cannot amortise transport costs the way a large aggregator can. Finally, language and digital-accessibility barriers limit the practical reach of many existing digital tools for the same farmers these tools are meant to serve.

Digital technology offers a credible path to narrowing these gaps, but only if it goes beyond digitising existing processes (as, for example, eNAM digitises the mandi auction) toward providing genuine decision support: taking the data that already exists and turning it into an answer to the question a farmer actually has — what should I sell, where, when, to whom, and how much will I actually make after costs. This is the specific need articulated in SIH problem statement PS-26132, and it is the need that motivates the design of CropWise.

CropWise is a full-stack digital platform built to address this need directly. Rather than functioning as a static mandi-price display application, CropWise combines live-capable market price intelligence, an explainable net-profit calculation that accounts for transport and handling costs, a rule-based selling-recommendation engine, a farmer-to-buyer marketplace with verification and competitive offers, buyer matching ranked by expected outcome, shared-transport and group-selling tools, and multilingual text/voice access. The remainder of this paper describes the problem in more detail, positions CropWise against the existing digital-agriculture landscape, and documents the system's architecture, implementation, and current capabilities and limitations in detail.

2. Problem Statement

PS-26132, titled "Strengthening Market Linkages and Price Discovery for Farmers," is a Software-track problem statement under the Agriculture, FoodTech & Rural Development theme of SIH 2026, sponsored by the Government of Maharashtra (Maharashtra State Innovation Society). The title decomposes into two distinct but linked failures that a solution must address.

2.1 Price Discovery Failure

Price discovery, in this context, refers to a farmer's ability to know — before selling — what their produce is actually worth right now, in enough nearby markets to make a real choice. India's mandi/APMC system is structurally fragmented: there are an insufficient number of regulated markets relative to farmer density, market infrastructure is uneven, intermediation costs are high, and information asymmetry forces farmers to physically travel to a mandi to learn its price, with no easy way to compare it against the next mandi over before making the trip. eNAM, the government's flagship digital fix launched in 2016, was built to solve this by creating a unified electronic market, but independent adoption studies converge on recurring barriers: farmers need immediate cash and distrust online payment, digital literacy and smartphone access are uneven, distance to the nearest eNAM-enabled mandi matters more than internet access, and grading/quality-assay infrastructure is too thin to let a remote buyer trust what they are bidding on. Crucially, eNAM does not perform the "should I sell now or wait" or "which market nets me more after transport" reasoning — it digitises the existing per-mandi auction rather than comparing markets for the farmer or accounting for transport cost in the price shown.

2.2 Market Linkage Failure

Market linkage refers to the connection — or lack of it — between a farmer with produce and the buyer who would pay the most for it. The literature identifies structural resistance from traditional intermediaries, restrictive state APMC Acts that historically limited seamless inter-state buyer-farmer matching, and the difficulty of assaying produce quality remotely, which is why traders overwhelmingly still prefer to inspect produce in person before buying. On the buyer and aggregation side, well-funded private platforms have already built large-scale farmer-buyer linkage businesses precisely because the public system does not close this gap; market linkage is estimated by industry sources to be the single largest value pool in Indian agritech, spanning warehousing, logistics, financing, and digital marketplaces.

2.3 Compounding Constraints

- Uncertain crop prices across nearby markets and over time, with no single reliable, farmer-facing comparison.
- Limited access to consolidated, decision-ready market information (as opposed to raw published price lists).
- Difficulty identifying and vetting suitable buyers, particularly for a first-time or remote transaction.
- Information asymmetry between farmers, traders, and buyers regarding both price and produce quality.
- Fragmented agricultural services — price data, buyer discovery, transport planning, and quality assessment are rarely available from one connected source.
- Transportation and logistics constraints that are frequently invisible in the price a farmer is shown, despite materially changing net proceeds.
- Quality uncertainty that discourages remote or first-time buyers from bidding competitively.
- Language and accessibility barriers that exclude a large share of the intended user base from existing digital tools.

CropWise's existing framing operationalises this problem statement as a single compound question a farmer actually has: What should I sell? Where? When? To whom? And how much will I actually make? This framing, and the system built around it, are described in the sections that follow.

3. Objectives

Based on the problem statement and the actual scope implemented in CropWise, the objectives of this work are:

1. To improve agricultural price discovery by presenting market prices across nearby markets alongside a transparent net-profit calculation, rather than a bare sticker price.
2. To connect farmers with suitable, verifiable buyers through a farmer-facing marketplace with server-side validated offers.
3. To provide market and arrival price intelligence sourced from government open data where available, with an honestly labelled fallback to seeded demonstration data.
4. To support informed selling-timing decisions through an explainable, rule-based sell-now-versus-hold recommendation engine.
5. To manage crop lots (harvest listings) digitally, from posting through offer negotiation to acceptance.
6. To facilitate group/cooperative selling and shared transport among nearby farmers to reduce per-unit logistics cost.
7. To provide AI-assisted decision support that is explainable by design rather than a black box, with every contributing factor shown to the farmer.
8. To improve accessibility through multilingual text support (15 fully supported languages, 38 selectable) and voice interaction (verified reliable in English and Hindi).
9. To provide administrators with live-computed platform insight (an impact dashboard and user-activity/login tracking) rather than static reporting.

4. Existing System and Literature Review

This section situates CropWise against the government digital-market infrastructure it builds on top of, the documented adoption literature around that infrastructure, and the private-sector agritech platforms that currently occupy the market-linkage space.

4.1 Government Digital Market Infrastructure

The National Agriculture Market (eNAM), launched by the Government of India in 2016, created a pan-India electronic trading portal that networks existing APMC mandis to provide a unified national market for agricultural commodities. It was built precisely to address mandi fragmentation and information asymmetry by digitising the mandi auction and making price and arrival data more broadly visible. AGMARKNET and the open-data portal data.gov.in similarly publish daily mandi price and arrival data for a wide range of commodities and markets across India, giving researchers and developers a raw, machine-readable price data source.

4.2 Documented Adoption Barriers

Independent assessments of eNAM adoption across states including Uttar Pradesh, Punjab, and Tamil Nadu converge on a consistent set of barriers: farmers' preference for immediate cash payment and distrust of online payment settlement; uneven digital literacy and smartphone access among the farmer population; the finding that physical distance to the nearest eNAM-enabled mandi matters more to actual usage than general internet availability; and thin grading/quality-assay infrastructure that leaves remote buyers unable to trust what they are

bidding on sight-unseen. These findings collectively point to a gap that is not "no digital price data exists" — AGMARKNET and data.gov.in already publish it — but that raw price lists are not decision support. A farmer comparing five mandi prices must still mentally compute (price − transport − mandi fee − handling) × risk of quality rejection for each option, typically without reliable transport-cost knowledge.

4.3 Private-Sector Agritech Platforms

A well-funded private-platform landscape has emerged specifically because the public system does not close the market-linkage gap. The following table summarises representative players and what each does and does not solve, as background context for positioning CropWise; this is a literature-informed overview rather than a first-party audit of each competitor's current systems.

Player	What It Solves	What It Does Not Solve (Positioning Gap)
eNAM (Govt.)	Digitises mandi auctions; provides national price visibility	No net-realisation math, no transport-aware comparison, no explainable "why", limited quality-trust infrastructure for remote bidding
AGMARKNET / data.gov.in	Publishes raw historical and daily mandi price data	Data only — no decision layer, no per-farmer context, no buyer linkage
DeHaat	Full-stack agri-services: inputs, advisory, credit, and market access via an agent network	Agent/franchise-dependent model, not a self-serve transparent price-discovery tool for an individual farmer
Ninjacart / WayCool / Arya.ag	B2B farm-to-retail logistics and produce sourcing at scale	Built for enterprise buyers; not oriented around helping an individual smallholder decide where to sell
AgroStar	Multilingual agri-input marketplace and advisory (Maharashtra-focused)	Input (purchasing) side of the farm business, not the output/selling side
Bijak	Transparent B2B price discovery for commodity traders	Trader-to-trader, not farmer-facing

4.4 Related Research Themes in Digital Agriculture

Beyond specific platforms, the broader digital-agriculture literature discusses recurring themes directly relevant to this project: (i) agricultural marketplace design and the trust/verification mechanisms needed for remote transactions; (ii) recommendation and decision-support systems for farm-level economic decisions, including the trade-off between model accuracy and explainability for a user base that may not trust an unexplained recommendation; (iii) price forecasting approaches ranging from simple trend/volatility models to more complex statistical and machine-learning methods, and the data volume such methods require to be reliable; and (iv) digital inclusion research documenting that language, literacy, and connectivity — not merely the existence of a digital tool — determine actual reach among smallholder farmer populations. These themes directly informed CropWise's design choices, discussed in Section 6 onward.

5. Research Gap

Synthesising Sections 2 and 4, no single existing system — public or private — combines all of the following in one connected, farmer-facing system: (a) transparent, sourced government price data; (b) an explainable net-realisation calculation that includes real transport economics rather than a bare sticker price; (c) buyer verification and demand signals; and (d) a self-serve, multilingual (including voice) interface aimed at an individual smallholder rather than an enterprise buyer or a trader network. eNAM and AGMARKNET solve data publication but not decision support or buyer linkage. Enterprise-oriented private platforms (Ninjacart, WayCool, Arya.ag) solve B2B logistics and sourcing at scale but are not designed around an individual smallholder's selling decision. Input-side platforms (AgroStar) and trader-facing platforms (Bijak) solve adjacent but distinct problems. This combination — done credibly, with each layer's provenance honestly labelled — is the specific gap CropWise is designed to occupy, and it matches how the underlying architecture of the system has been built.

A second, narrower research gap concerns explainability in agricultural decision support. Where AI or algorithmic recommendation is introduced into a farmer-facing tool, the adoption-barrier literature (Section 4.2) suggests that trust — not raw predictive accuracy — is often the binding constraint on usage, particularly for a first-time or low-literacy user. This motivates CropWise's architectural decision, discussed in Section 15, to implement its recommendation and forecasting logic as transparent, explainable rule-based and statistical models rather than as opaque machine-learning models, at least at the current stage of the system's development.

6. Proposed System — CropWise

CropWise is proposed and implemented as an integrated digital platform that brings together, within one authenticated application, the layers that the existing landscape (Section 4) leaves separated. The system's pre-login positioning statement — "Find the Right Market. Get the Right Price." — and its core tagline — "Smart Markets. Better Prices. Stronger Farmers." — reflect this integration goal.

Concretely, the platform brings together the following capabilities under one farmer or buyer login:

- Market Intelligence: live-style price comparison across nearby markets with a full net-profit breakdown (price minus transport, mandi charges, and handling), rather than the sticker price alone.
- AgriAdvisor: an explainable AI selling recommendation — a sell-now-versus-hold percentage split — with every contributing factor (demand, supply, weather, transport, price trend) shown rather than hidden.
- Price Forecast: a 7-day price forecast built from a transparent trend-and-volatility model, with a visible confidence score and chart.
- AgriMarket: a farmer-to-buyer marketplace where farmers post harvest listings and verified buyers browse and submit competing offers in a reverse-auction style, with server-side validation.
- Smart Buyer Matching: buyers ranked for a given listing by estimated net profit, reliability, payment history, distance, and crop interest, with the reasons for the ranking shown.
- FarmPool: a shared-transport calculator that pools a farmer's shipment with nearby farmers heading to the same market and shows the resulting savings.
- Profit Calculator: a side-by-side comparison of multiple selling scenarios (for example, local mandi versus a direct buyer versus a more distant market).
- Group Selling: FPO/cooperative pooling with per-farmer membership tracking.

- Alerts: price-drop, high-demand, opportunity, and harvest-reminder notifications.
- Multilingual Farm Assistant: a text-based assistant with a language-neutral intent/entity engine usable in any of 15 fully supported UI languages, with reliable voice input/output in English and Hindi.
- AI-Ready Quality Assessment: a deterministic quality-grading service that validates the marketplace workflow end-to-end, structured so a trained computer-vision model can later replace it without changing the marketplace APIs.
- Impact Dashboard and User Activity tracking: admin-authenticated, live-computed platform metrics and login-activity monitoring.

A guiding design principle throughout the system is honesty of data provenance: every price record, forecast, and recommendation is labelled as being backed by live government data or by seeded demonstration data, and the two are never silently blended. This principle, and the corresponding architectural pattern used to enforce it, are discussed further in Sections 10 and 15.

7. Working Principle

This section explains, step by step, how a farmer's use of CropWise proceeds from account access to a completed transaction, and correspondingly for a buyer. Both workflows described below are implemented and functional in the current codebase; no step in this section is a proposed or prototype-only workflow.

7.1 Farmer Working Principle

A farmer's interaction with CropWise follows the sequence: account registration or JWT-authenticated login, arrival at the farmer dashboard, posting a crop/harvest listing on AgriMarket (or, independently of listing, consulting Market Intelligence for an unlisted crop), comparing prices and computing net profit across nearby markets, optionally consulting AgriAdvisor for an explainable sell-now-versus-hold recommendation and the Price Forecast module for a 7-day outlook, reviewing buyer offers or smart buyer matches on an active listing, and finally accepting an offer (individually or via FarmPool/Group Selling for shared logistics) to complete the transaction. Figure 2 illustrates this sequence.

Figure 2. Farmer User Workflow in CropWise

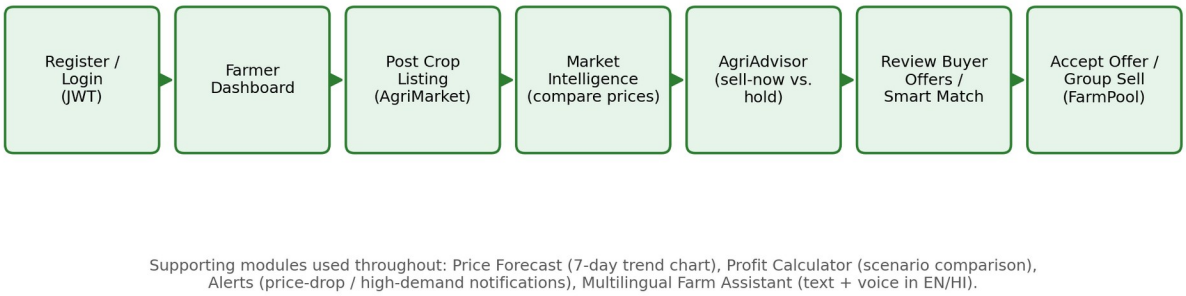

Figure 2. Farmer user workflow in CropWise, from authentication through to a completed sale.

7.2 Buyer Working Principle

A buyer's interaction proceeds through registration and verification, arrival at the buyer dashboard, browsing active AgriMarket listings, reviewing a listing's details and quality grade, submitting a competing offer (subject to server-side validation), and — pending the farmer's decision — having the offer accepted or rejected, after which a transaction record is created. Figure 3 illustrates this sequence.

Figure 3. Buyer User Workflow in CropWise

Figure 3. Buyer user workflow in CropWise, from registration through to a completed transaction.

7.3 Implemented vs. Proposed Distinction

Both workflows above describe functionality present in the current implementation, running on seeded demonstration data by default with an available live-data path for market prices (Section 10.8). Extensions that are not yet implemented — for example, machine-learning-based price forecasting, satellite/IoT-derived crop and weather signals, or a trained computer-vision quality model — are explicitly deferred to Section 22 (Future Scope) and are not part of the working principle described here.

8. System Architecture

CropWise follows a layered, client-server architecture with a clear separation between presentation, API, business logic, and data layers. Figure 1 shows the overall architecture as implemented.

Figure 1. Overall System Architecture of the CropWise Platform

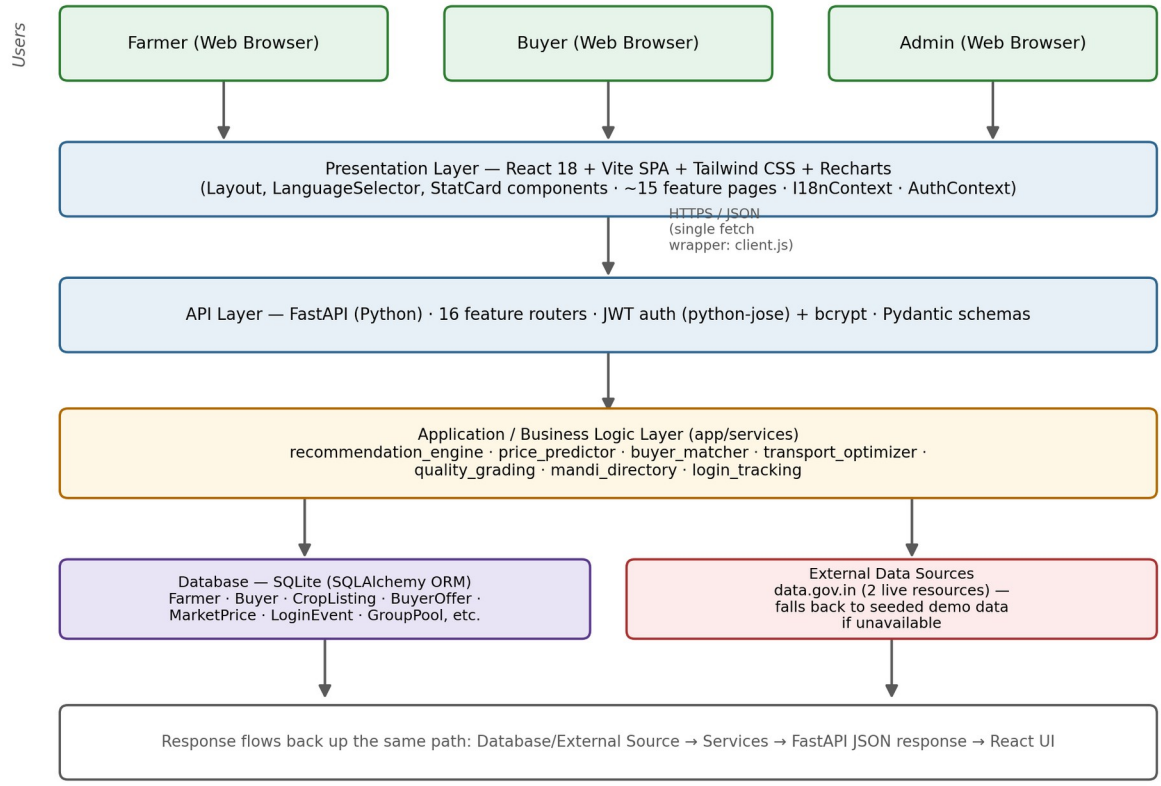

Figure 1. Overall architecture of the CropWise platform.

8.1 Presentation Layer

The presentation layer is a React 18 single-page application built with Vite and styled with Tailwind CSS, using Recharts for data visualisation (price trend and forecast charts). It is organised as one page per feature area (approximately 15 pages: dashboard, market intelligence, AgriAdvisor, price forecast, AgriMarket, buyer dashboard, buyer verification, buyer demands, FarmPool, profit calculator, group selling, alerts, the multilingual assistant, admin/impact dashboard, and authentication). Shared components include a layout shell, a language selector, and reusable stat-card components. Two React contexts — AuthContext and I18nContext — manage authentication state and language/localisation state respectively across the application, and a single fetch wrapper (client.js) centralises all API communication.

8.2 API Layer

The API layer is implemented in Python using FastAPI, organised as sixteen feature-area routers (one router per functional module, for example market, assistant, marketplace/AgriMarket, buyer verification, admin, and authentication routers). Requests are authenticated via JSON Web Tokens (JWT, via python-jose) with bcrypt password hashing, and validated against Pydantic schemas before reaching business logic.

8.3 Application / Business Logic Layer

Business logic is concentrated in a services module (app/services) rather than in the routers themselves, so that logic such as market comparison, price prediction, buyer matching, transport optimisation, quality grading, mandi/market-name discovery, and login-event tracking is implemented once and shared across every router and page that needs it. This is a deliberate architectural choice that avoids duplicated logic between, for example, the Market Comparison page, the Profit Calculator, and FarmPool, all of which share the same underlying transport_optimizer module.

8.4 Data Layer

The data layer consists of a SQLite database accessed through the SQLAlchemy ORM, chosen for zero-configuration local and demo deployment, alongside an external data layer comprising two data.gov.in government open-data resources for live mandi price data. The mandi_directory service sits in front of both live resources and the internal database, performing market-name discovery and fuzzy matching so that a local town name can be resolved to the government's exact market or district naming convention (see Section 10.8).

8.5 Cross-Cutting Concerns

Two concerns span every layer rather than belonging to a single one: multilingual support (Section 10.19), implemented so that language is strictly a presentation-layer concern and never forks business logic; and data-source honesty, implemented so that every price, forecast, or recommendation response is explicitly labelled as being backed by live or demo data rather than silently blended.

9. Technology Stack

Table 2 documents the technology stack as declared in the project's own README and dependency configuration, rather than a stack inferred or assumed to be suitable.

Layer	Technology	Purpose
Frontend framework	React 18 + Vite	Single-page application shell, component rendering, fast development build/bundling
Frontend styling	Tailwind CSS	Utility-first responsive styling across ~15 feature pages
Data visualisation	Recharts	Price trend and 7-day forecast charts
Backend framework	Python + FastAPI	REST API layer, request routing, automatic interactive API docs (/docs)
ORM / Database access	SQLAlchemy	Object-relational mapping between Python models and the database
Database	SQLite	Zero-configuration, file-based relational database (swappable for PostgreSQL via DATABASE_URL)
Authentication	JWT (python-jose) + bcrypt	Token-based session authentication and salted password hashing
Multilingual (i18n)	Custom lightweight i18n module	Language-neutral intent/entity extraction and native-language response templates (no external translation/LLM API required)

Layer	Technology	Purpose
Voice	Browser-native Web Speech API	Speech-to-text (SpeechRecognition) and text-to-speech (SpeechSynthesis) behind a swappable provider interface
External data source	data.gov.in Open Government Data API	Live daily mandi commodity price and district/variety-wise market price resources
Version control	Git / GitHub	Source control and collaboration (repository: github.com/Abha153/cropwise)
Deployment (frontend)	Vercel	Hosting for the live frontend deployment (cropwise-alpha.vercel.app)

Note: the backend deployment target referenced in the project documentation is Render, with an explicit operational caveat documented in Section 16.6 regarding SQLite persistence on Render's default ephemeral disk. No machine-learning framework (for example scikit-learn, TensorFlow, or PyTorch) is declared as a dependency in the current build; the decision-support and forecasting logic is implemented directly in Python as rule-based and statistical models (Section 15).

10. Functional Modules

This section documents every major functional module in CropWise: its purpose, working, user interaction, backend/data interaction, and implementation status. Status labels used throughout follow the classification IMPLEMENTED, PARTIALLY IMPLEMENTED, PROTOTYPE, SEEDED/FALLBACK DATA, EXTERNAL DEPENDENCY, and PROPOSED/FUTURE, as defined in Section 0 of this paper's methodology and used consistently in Sections 17–22.

10.1 Farmer Module

Status: IMPLEMENTED

Purpose: Provides farmer account registration, authentication, and a personalised dashboard as the entry point to all farmer-facing features.

Working: A farmer registers or logs in (including via pre-seeded demo accounts, e.g. ramesh@cropwise.demo) and is issued a JWT on successful authentication.

User Interaction: Farmers access the dashboard, post listings, view market data, and interact with AgriAdvisor, Price Forecast, and FarmPool from this identity.

Backend/Data: Backed by the Farmer SQLAlchemy model (name, phone, email, location, password_hash, last_login) and JWT-guarded FastAPI endpoints; public endpoints return a PII-free schema, with contact details only in authenticated /me responses.

10.2 Buyer Module

Status: IMPLEMENTED

Purpose: Provides buyer account registration, authentication, and a dashboard for browsing listings and managing offers.

Working: A buyer registers, logs in, and gains access to AgriMarket browsing, offer submission, and buyer-demand posting.

User Interaction: Buyers use dedicated dashboard pages and offer forms; only authenticated buyers can submit or manage offers.

Backend/Data: Backed by the Buyer SQLAlchemy model and BuyerOffer relations; offer submission is server-side validated (Section 10.6).

10.3 Buyer Verification

Status: IMPLEMENTED

Purpose: Establishes buyer trustworthiness signals so that farmers can assess who they are dealing with before accepting an offer.

Working: Dedicated verification pages and routes exist in the codebase (BuyerVerification.jsx on the frontend, a buyer verification router and buyer_matcher service on the backend).

User Interaction: Farmers can view a buyer's verification status/profile alongside an offer; buyers may need to complete verification steps to be included in Smart Buyer Matching.

Backend/Data: Implemented as dedicated frontend pages/routers and a backend matcher service; a full line-by-line audit of exactly which verification checks are enforced has not been performed as part of this paper (see Section 21).

10.4 Buyer Demands

Status: IMPLEMENTED

Purpose: Lets buyers signal the crops and quantities they are seeking, independent of a specific farmer listing, to aid discovery in both directions.

Working: Buyers post demand records (crop, quantity, price range, location) via a dedicated BuyerDemands page and router.

User Interaction: Farmers can browse posted buyer demand to understand what is being sought locally before or instead of posting a listing.

Backend/Data: Backed by a dedicated demand data model and router (buyer_matcher.py / BuyerDemands.jsx); whether the demand shown is clearly labelled as a CropWise-derived signal versus an implied official figure has not yet been audited (Section 21).

10.5 Marketplace (AgriMarket)

Status: IMPLEMENTED

Purpose: The core transactional venue: farmers post harvest listings, and verified buyers browse and submit competing (reverse-auction style) offers.

Working: A farmer creates a listing with crop, quantity, unit, and a minimum acceptable price; buyers view active listings and submit offers, which the farmer can accept or reject.

User Interaction: Farmers view "My Listings" and offers received; buyers browse listings and submit offers from the AgriMarket page.

Backend/Data: Backed by the CropListing and BuyerOffer models; server-side validation enforces positive price/quantity, offer quantity not exceeding the listing, rejection of offers below the farmer's minimum price, and prevents double accept/reject on an inactive listing.

10.6 Farmer–Buyer Matching (Smart Buyer Matching)

Status: IMPLEMENTED

Purpose: Ranks candidate buyers for a given listing so a farmer does not have to manually evaluate every offer against every factor.

Working: The buyer_matcher service scores buyers by estimated net profit, reliability, payment history, distance, and crop interest, and returns the ranking with reasons shown.

User Interaction: A farmer opens "Smart buyer matches" on an active listing and sees a ranked list rather than a flat offer feed.

Backend/Data: Requires the listing owner's authenticated farmer login (a documented security fix — this endpoint was previously unauthenticated and is now access-controlled).

10.7 Crop Lot Management

Status: IMPLEMENTED

Purpose: Manages the lifecycle of a farmer's harvest listing from creation through active bidding to a final accepted/rejected/closed state.

Working: Listings carry crop, quantity, unit, minimum acceptable price, quality grade, and an active/inactive status that governs whether new offers can be submitted.

User Interaction: Farmers create, view, and manage their own listings; the status transitions are enforced so an inactive listing cannot receive further offers.

Backend/Data: Backed by the CropListing model and associated CRUD endpoints in the marketplace router.

10.8 Mandi Price Intelligence (Market Intelligence)

Status: PARTIALLY IMPLEMENTED / SEEDED + LIVE-CAPABLE

Purpose: Provides live-style price comparison across nearby markets together with a full net-profit breakdown (price minus transport, mandi charges, and handling).

Working: The page queries live_market_data.py and district_market_data.py for government price data when live mode is enabled, falling back transparently to sixty days of seeded, deterministic synthetic price data otherwise; every price row is tagged data_source: "live" or "demo".

User Interaction: A farmer selects a crop and compares candidate markets; the recommended market and the real net profit after transport are shown, with a 🟢 Government Data / 🟡 Demo Data badge per row.

Backend/Data: A verified implementation bug exists in the live-fetch path: it currently filters on filters[state] where the correct data.gov.in resource field is state.keyword, confirmed against the live OpenAPI Specification; this and related district/market/commodity/variety/grade filters require a full filter-name audit (see Section 21).

10.9 Arrival Intelligence

Status: PARTIALLY IMPLEMENTED

Purpose: Surfaces market arrival volumes alongside price, since arrival quantity is a leading indicator of near-term price movement.

Working: Arrival data is sourced from the same live/demo data pipeline as Market Intelligence (Section 10.8), integrated into the recommendation engine's supply-side signal.

User Interaction: Arrival trends inform the AgriAdvisor recommendation rather than being exposed as a fully separate, dedicated page in the current build.

Backend/Data: Same live/demo data-source labelling and known filter-field caveat as Section 10.8 applies.

10.10 Quality Assessment

Status: PROTOTYPE

Purpose: Establishes a quality grade for a crop listing so that buyers can trust what they are bidding on without physically inspecting it, addressing the remote-quality-assay gap identified in Section 4.2.

Working: The quality_grading.py service deterministically simulates a computer-vision grading result from the crop name/image filename rather than analysing actual image content.

User Interaction: A quality grade is attached to and displayed on a listing, validating the full marketplace workflow (grading → listing → matching) end-to-end.

Backend/Data: The service boundary is explicitly designed so a trained image-classification model can later be substituted without changing the marketplace APIs; it is intentionally not presented as a live computer-vision model. Image uploads for grading are simulated by filename only — no actual file upload/storage is wired up in the current build.

10.11 Group Selling / FarmPool

Status: IMPLEMENTED

Purpose: Two related cooperative-selling features: FarmPool computes shared-transport savings when a farmer's shipment is pooled with nearby farmers heading to the same market; Group Selling supports FPO/cooperative-style pooling of produce with per-farmer membership tracking.

Working: FarmPool draws on the shared transport_optimizer service (also used by Market Comparison and the Profit Calculator) to compute pooled versus individual transport cost. Group Selling membership records update the farmer's own row on re-joining a pool rather than summing on top of a previous join.

User Interaction: A farmer views potential pooling partners and the resulting savings, or joins/updates their membership in a cooperative selling pool.

Backend/Data: Backed by a pool/membership data model and the shared transport_optimizer service; a documented security fix ensures pool membership no longer double-counts on re-join.

10.12 AI Advisor (AgriAdvisor)

Status: **IMPLEMENTED (rule-based, explainable — not a trained ML model)**

Purpose: Provides a farmer-facing selling-timing recommendation (sell-now versus hold, expressed as a percentage split) with full transparency into why the recommendation was made.

Working: The recommendation_engine.py service combines demand, supply, a deterministic simulated weather-risk signal, price trend, and transport cost into a factor-scored, rules-based recommendation, per the project's own documented architecture (README §22).

User Interaction: A farmer requests a recommendation for a crop/quantity/location and receives a sell-now/hold split alongside every contributing factor, rather than a single opaque score.

Backend/Data: Implemented as an explainable rules engine over real price/cost data rather than a general-purpose large language model, by explicit design choice; UX presentation as structured response cards versus paragraphs has not yet been fully audited (Section 21).

10.13 Price Forecasting

Status: **IMPLEMENTED (transparent statistical model — not machine-learned)**

Purpose: Produces a 7-day price forecast with a visible confidence score, to support the sell-now-versus-hold decision in AgriAdvisor.

Working: price_predictor.py implements a transparent linear-trend plus volatility model over the available (live or demo) historical price series.

User Interaction: The forecast is shown as a chart with a confidence score on the Price Forecast page and feeds into AgriAdvisor's recommendation.

Backend/Data: The project's own documentation states the return shape is designed to be ready to swap in a trained model (for example Prophet, XGBoost, or an LSTM) without changing the API or frontend contract — this swap has not been made in the current build.

10.14 Storage

Status: **PROPOSED / NOT IMPLEMENTED AS A DEDICATED MODULE**

Purpose: Storage-related decision support (for example cold-storage recommendations or spoilage-risk estimation) is part of the broader PS-26132 problem space but is not implemented as a distinct CropWise module in the current codebase.

Working: Not applicable in the current build.

User Interaction: Not applicable in the current build.

Backend/Data: No storage-specific service, router, or page was identified in the project's own documentation; this is flagged as a gap for future work (Section 22) rather than described as implemented.

10.15 Transportation / Logistics

Status: IMPLEMENTED

Purpose: Provides transport-cost-aware decision support so that a market comparison, profit calculation, or pooling decision reflects real logistics cost rather than price alone.

Working: The shared transport_optimizer.py service is used identically across Market Comparison, the Profit Calculator, and FarmPool, computing distance-based transport cost that is subtracted from the sale price to reach a net-profit figure.

User Interaction: Transport cost appears as a line item in the net-profit breakdown on Market Intelligence, Profit Calculator, and FarmPool.

Backend/Data: A verified limitation exists here: transport costing currently uses a flat ₹7.5-per-kilometre-per-tonne rate plus a ₹60–150 base fee, with no diesel-price variable, no vehicle-class differentiation, and no minimum-trip economics — the same unrealistic-transport-figures issue the project's own specification flags (Section 21).

10.16 Transactions

Status: IMPLEMENTED

Purpose: Records the outcome of an accepted offer as a completed transaction for both the farmer's and the platform's records.

Working: When a farmer accepts a BuyerOffer, the offer's status transitions to accepted and the associated listing is closed to further offers.

User Interaction: Farmers and buyers can see accepted-offer/transaction status from their respective dashboards.

Backend/Data: Backed by status fields on the BuyerOffer/CropListing models rather than a separate ledger/payments table; no payment-processing integration is implemented.

10.17 Notifications (Alerts)

Status: IMPLEMENTED

Purpose: Surfaces time-sensitive information the farmer would otherwise have to check for manually.

Working: The Alerts module generates price-drop, high-demand, opportunity, and harvest-reminder notifications from the underlying market and listing data.

User Interaction: Farmers view alerts from a dedicated Alerts page.

Backend/Data: Generated from existing market/listing data rather than a separate real-time push/messaging infrastructure (for example no SMS/WhatsApp integration is implemented).

10.18 Admin Dashboard

Status: IMPLEMENTED

Purpose: Gives platform administrators live-computed visibility into platform usage and impact, separate from any farmer or buyer account.

Working: The Impact Dashboard (admin-authenticated) computes farmers/buyers connected, transaction counts, transport savings, and estimated additional farmer income. The User Activity section separately reports registered-account counts versus unique users who have actually logged in, successful/failed login counts, and a recent-activity feed.

User Interaction: An administrator authenticates via a separate, server-side-only admin login (ADMIN_USERNAME/ADMIN_PASSWORD environment variables, never hard-coded or shipped in the frontend bundle) and views /admin.

Backend/Data: Backed by the require_admin role guard, the login_tracking service, and a new login_events table; login_events deliberately never stores passwords, hashes, tokens, API keys, IP addresses, or the raw email typed at login — only a numeric user id (when matched), role, timestamp, and success/failure.

10.19 Multilingual Support (i18n)

Status: IMPLEMENTED (core architecture) / PARTIALLY VERIFIED (breadth)

Purpose: Makes the platform's UI and assistant usable across a wide range of Indian and international languages, addressing the accessibility barrier identified in Section 2.3.

Working: Fifteen languages are fully supported (UI translated, assistant intent/entity extraction, native-language response templates); a further twenty-three languages are selectable in the UI with an honest English-fallback badge for AI understanding and response. Language is architected strictly as a presentation-layer concern: one recommendation/matching/validation engine serves every language.

User Interaction: A farmer or buyer selects a language via the LanguageSelector component (present on every page); the assistant asks for clarification rather than silently guessing an unrecognised crop or location.

Backend/Data: The full, machine-readable capability matrix is served live at GET /assistant/languages and mirrored in backend/app/i18n/languages.py; the README documents that voice input/output is reliable only in English and Hindi, and that a full hard-coded-string audit across all frontend pages has not yet been completed (Section 21).

10.20 Voice Assistant

Status: IMPLEMENTED (English and Hindi verified) / DEVICE-DEPENDENT

Purpose: Provides spoken (rather than typed) interaction with the Multilingual Farm Assistant for lower-literacy or hands-busy usage contexts.

Working: Voice input uses the browser-native Web Speech API (SpeechRecognition) with no external speech-to-text API key required; voice output uses the browser's SpeechSynthesis API, checked live per language/device via speechSynthesis.getVoices() before a speaker icon is ever shown as active.

User Interaction: A farmer taps the microphone button (verified in Chrome) and speaks a query; if a language lacks a voice on the user's device, CropWise reports this rather than failing silently.

Backend/Data: Explicitly scoped to English and Hindi for reliable speech-to-text, by design, because Web Speech API recognition quality for most Indian regional languages is inconsistent across browsers/operating systems in practice; other languages retain full UI/text/assistant support without a microphone button.

11. User Workflow

The detailed step-by-step farmer and buyer workflows, and their corresponding workflow diagrams (Figures 2 and 3), are presented in Section 7 (Working Principle) to avoid duplicating the same diagrams and narrative twice in this paper. Section 7 should be read as the authoritative reference for both user workflows.

12. Data Flow Diagram

Figure 4 presents a level-1 data flow diagram for a representative CropWise request — for example, a Market Intelligence price comparison. The user's action in the React frontend triggers a single fetch-wrapper call to the FastAPI backend; the relevant router authenticates and validates the request and delegates to a business-logic service; the service reads from the SQLite database and, where live mode is enabled and applicable, additionally queries the external data.gov.in API, merging or falling back between the two as described in Section 10.8; the service returns a structured, source-labelled result up through the router as a JSON response; and the frontend renders this response as charts, tables, or forms for the user.

Figure 4. Data Flow Diagram (Level-1) for a Typical CropWise Request

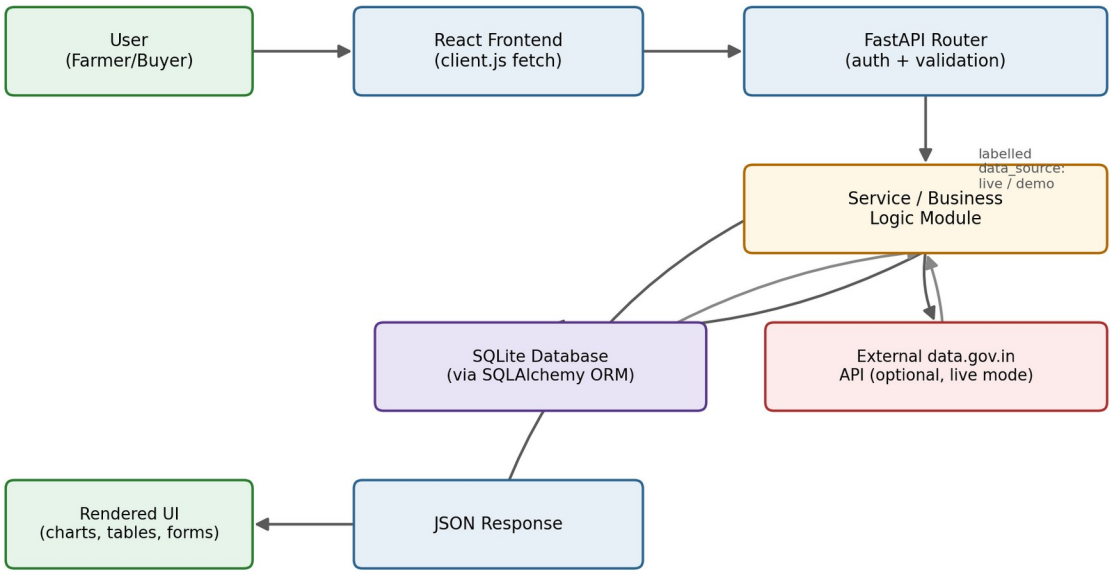

Figure 4. Level-1 data flow diagram for a typical CropWise request.

13. API Architecture

CropWise's backend exposes a REST API organised into sixteen feature-area FastAPI routers, mirroring the sixteen functional modules described in Section 10 (authentication, farmer, buyer, marketplace/AgriMarket, buyer verification, buyer demands, buyer matching, market/price intelligence, assistant, FarmPool/group pooling, price forecast, quality grading, alerts, admin, and login-activity endpoints). Figure 5 shows the general request-processing flow followed by every authenticated endpoint.

Figure 5. API and Backend Request-Processing Flow

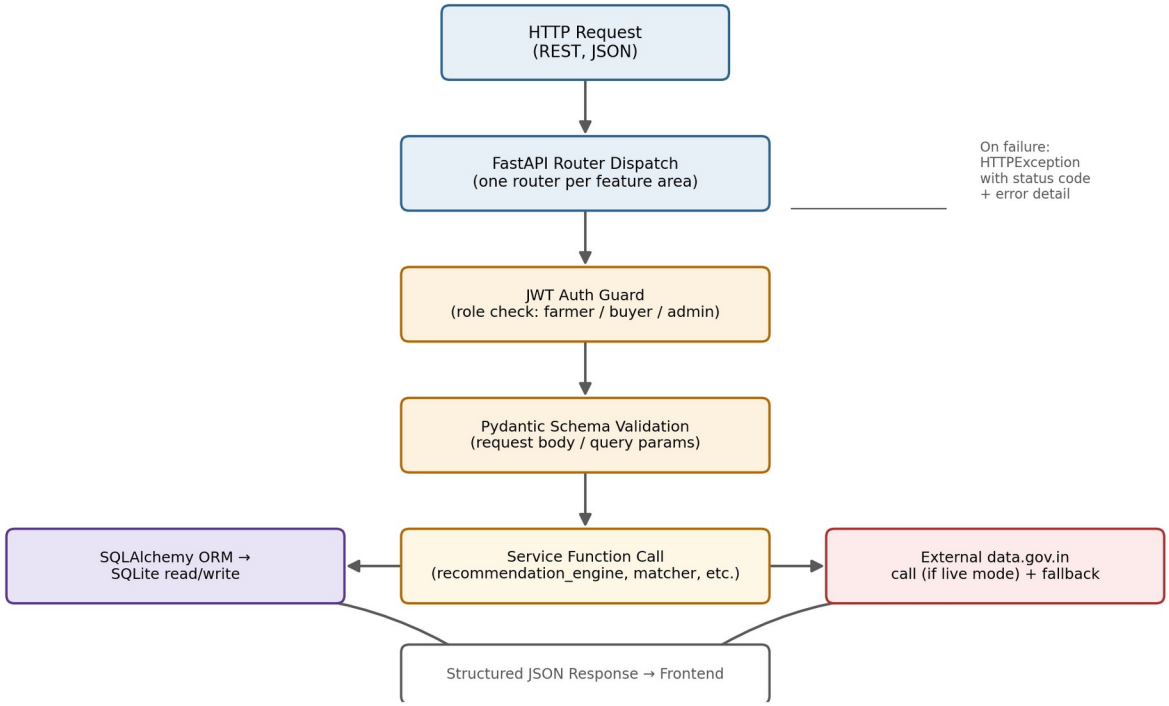

Figure 5. API and backend request-processing flow.

13.1 API Organisation

Each router is scoped to one feature area and delegates business logic to the corresponding service module (Section 8.3) rather than implementing logic inline, so the same computation (for example, transport cost or the recommendation score) is never duplicated across endpoints.

13.2 Authentication

Farmer and buyer authentication uses JWT bearer tokens (python-jose) with bcrypt-hashed passwords. Admin authentication is deliberately implemented as a separate, server-side-only credential path (ADMIN_USERNAME / ADMIN_PASSWORD environment variables), independent of the farmer/buyer JWT system, verified entirely server-side — the frontend admin login form only ever forwards typed credentials to POST /auth/admin/login and embeds no credentials of its own.

13.3 Validation and Error Handling

Requests are validated against Pydantic schemas before reaching business logic. Marketplace offer submission specifically enforces: positive price and quantity; offer quantity not exceeding the listing's remaining quantity; rejection of offers below the farmer's stated minimum acceptable price; and prevention of double accept/reject actions or actions against an inactive listing. On failure, endpoints return an HTTPException carrying an appropriate status code and error detail rather than a generic failure.

13.4 External API Integration

Market-price endpoints integrate with two data.gov.in Open Government Data resources: "Current Daily Price of Various Commodities from Various Markets" (resource ID 9ef84268-d588-465a-a308-a864a43d0070), keyed by an exact market name; and "Variety-wise Daily Market Prices Data of Commodity" (resource ID 35985678-0d79-46b4-9ed6-6f13308a1d24), keyed by revenue district. The mandi_directory service discovers actual government market names for a state, fuzzy-matches local town names against them (plus a small hand-maintained alias table), and merges the two resources — preferring the market-level hit but attaching district-level detail when both have data. This live path activates only when DATA_GOV_IN_API_KEY is set and MARKET_DATA_SOURCE=live is configured, and transparently falls back to the seeded demo dataset otherwise (Section 15.2 discusses the honesty labelling this produces).

13.5 Security Notes for This Documentation

No API keys, tokens, passwords, or .env values are reproduced anywhere in this paper, consistent with the project's own security practice of never committing or documenting real secret values (Section 23).

14. Database Design

CropWise persists data in a SQLite database accessed through SQLAlchemy models, chosen for zero-configuration setup; the project's own documentation notes that swapping DATABASE_URL for a PostgreSQL connection string is sufficient to move to a production-style database without changing the SQLAlchemy models. Figure 6 presents an entity-relationship overview of the major entities identified from the project's documented data model.

Figure 6. Entity-Relationship Overview of the CropWise Data Model

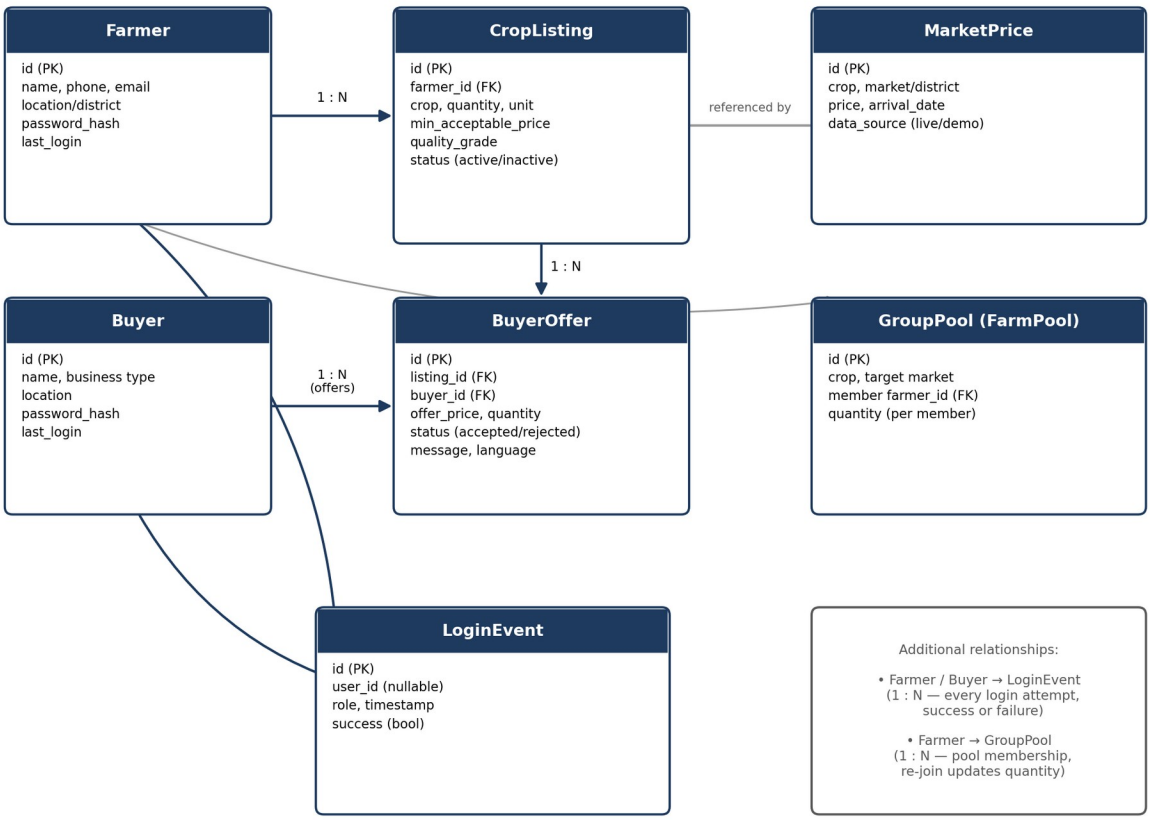

Figure 6. Entity-relationship overview of the CropWise data model.

14.1 Major Entities

- Farmer — a registered farmer account, holding contact and location details, a hashed password, and a last_login timestamp.
- Buyer — a registered buyer account (for example a processor, retailer, or exporter in the seeded demo data), structurally parallel to Farmer.
- CropListing — a farmer's harvest listing (crop, quantity, unit, minimum acceptable price, quality grade, and active/inactive status), the core object of the AgriMarket marketplace.
- BuyerOffer — a buyer's competing offer against a specific listing (offer price, quantity, status, and an optional message carrying its source language for the cross-language architecture described in Section 10.19).
- MarketPrice — a persisted price record for a crop/market or crop/district combination, explicitly tagged with a data_source of "live" or "demo" so that historical charts and forecasts only draw on genuine accumulated data by default.
- GroupPool (FarmPool) — a shared-transport or cooperative-selling pool, with per-farmer membership records that update in place on re-joining rather than double-counting.
- LoginEvent — an append-only record of every authentication attempt (farmer, buyer, or admin), storing only a numeric user id when matched, role, timestamp, and success/failure — by explicit design, never a password, hash, token, API key, IP address, or the raw email typed.

14.2 Migration Approach

Schema evolution is handled through Base.metadata.create_all() on startup followed by a small, additive-only lightweight migration routine that only ever adds a table or adds a nullable column, and never drops a table, drops a column, or rewrites existing row data — a property specifically relied upon to allow safe updates to a live deployment with existing user data.

15. AI/ML Methodology

This section documents, without overstatement, exactly what kind of decision-support logic CropWise implements. Based on the project's own architecture documentation and dependency list, CropWise does not currently employ a trained machine-learning model anywhere in its decision-support pipeline. Every recommendation, forecast, and matching score is produced by explainable rule-based logic or transparent statistical computation over real price and cost data. This is stated as a deliberate architectural decision in the project's own documentation, not as an omission.

15.1 AgriAdvisor: Rule-Based Recommendation Engine

The recommendation_engine.py service computes a sell-now-versus-hold percentage split by combining five inputs: demand, supply, a deterministic simulated weather-risk signal, price trend, and transport cost. Because this is rules-based rather than a trained model, each of the five contributing factors is shown alongside the final recommendation, so that the farmer sees why a recommendation was made rather than a bare percentage. This directly follows the project's stated design rationale (its own specification, §22) that an explainable engine over real data is architecturally preferable, for this user base, to a general-purpose large language model.

15.2 Price Forecast: Trend-and-Volatility Model

price_predictor.py implements a transparent linear-trend plus volatility statistical model over the available historical price series (live-accumulated or seeded-demo, clearly labelled) to produce a 7-day forecast with a visible confidence score. The output data shape is documented as being deliberately compatible with a future drop-in replacement by a trained model (for example Prophet, XGBoost, or an LSTM network) without requiring API or frontend changes; no such model has yet been trained or integrated.

15.3 Multilingual Intent/Entity Extraction

The assistant's natural-language understanding (app/i18n/nlu.py) is a keyword-based intent classifier plus rule-based entity extraction for crop, quantity, and location — not a general-purpose NLU model or large language model. This is fast, transparent, and has zero external dependency, and the project documents that it correctly handles the problem statement's example queries (in Marathi, Hindi, and Bhojpuri) while explicitly not being general-purpose NLU. Critically, when the extractor cannot identify a crop or location in the input, the system returns a clarification_needed response rather than defaulting to a guessed value — the project's own changelog records this as a direct fix to a prior version that defaulted unknown crops to "Tomato" and unknown locations to "Bilaspur."

15.4 Buyer Matching: Multi-Factor Scoring

The buyer_matcher.py service ranks candidate buyers for a listing using a weighted combination of estimated net profit, reliability, payment history, distance, and crop interest — a deterministic scoring function over stored/derived attributes, not a learned ranking model.

15.5 Quality Grading: Deterministic Simulation

The quality_grading.py service deterministically derives a simulated grading result from the crop name or an image filename string, rather than performing any actual image analysis. This is explicitly not presented as a live computer-vision model; its function is to validate the end-to-end marketplace workflow (grading → listing → matching) so that a trained image classifier can later be substituted behind the same service interface.

15.6 Weather Risk Signal

The weather-risk input to AgriAdvisor is a deterministic simulated signal within recommendation_engine.py, standing in for a live weather-data API integration that has not yet been added.

15.7 Summary of AI/ML Status

In summary: CropWise's "AI" functionality is decision support built from explainable rules and transparent statistics, not trained machine learning. This is documented here as an accurate description of the current system, and Section 22 discusses realistic paths toward introducing trained models (larger historical datasets, forecasting models such as Prophet/XGBoost/LSTM, and a trained vision model for quality grading) as clearly labelled future work rather than as claims about the present system.

Figure 7. AI-Assisted Decision-Support (AgriAdvisor) Flow

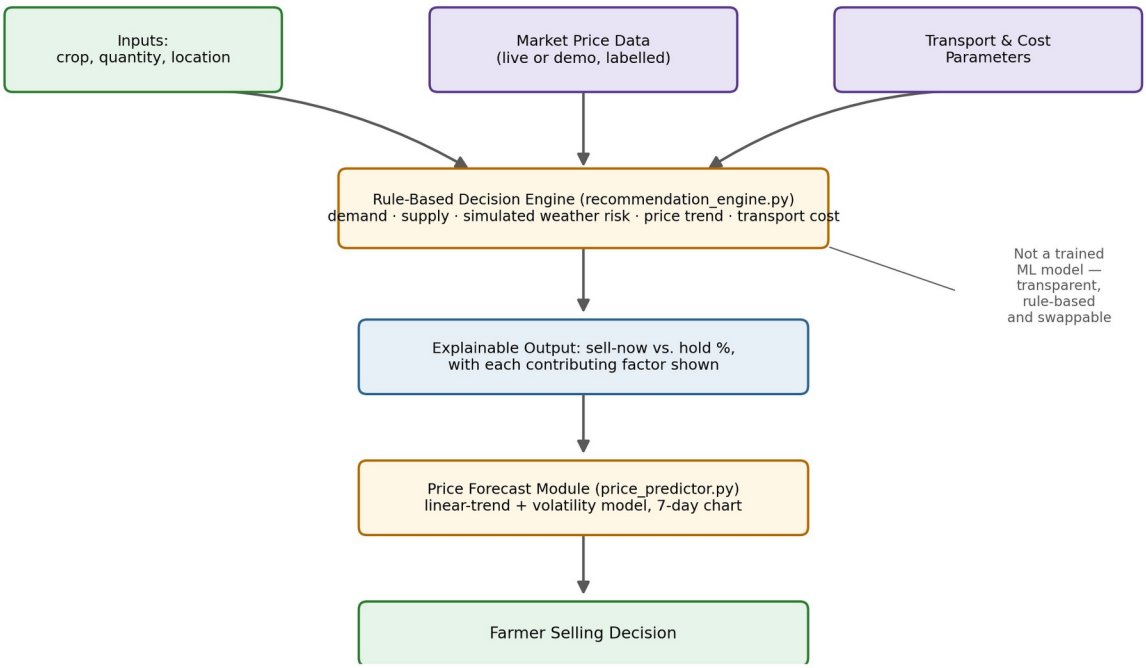

Figure 7. AI-assisted decision-support (AgriAdvisor) flow, showing the rule-based engine and its inputs/outputs.

16. Implementation

16.1 Frontend Implementation

The frontend is implemented as a React 18 + Vite single-page application with roughly fifteen feature pages, shared Layout/LanguageSelector/StatCard components, an AuthContext for authentication state, and an I18nContext that lazily loads per-language translation JSON files. All backend communication is centralised through a single fetch wrapper (src/api/client.js).

16.2 Backend Implementation

The backend is implemented as a Python FastAPI application (app/main.py wiring sixteen routers), with configuration (including admin credentials and the runtime JWT secret) in app/config.py, SQLAlchemy engine/session setup in app/database.py, ORM models in app/models.py, Pydantic request/response schemas in app/schemas.py, and JWT/password/role-guard utilities in app/auth_utils.py.

16.3 Database Implementation

app/seed_data.py seeds the SQLite database with demo farmers, buyers, listings, offers, and sixty days of historical prices for every crop/market pair, but only when the farmers table is completely empty — making seeding a safe no-op on every subsequent startup once any farmer exists, whether seeded or genuinely registered.

16.4 API Integration

External API integration is implemented via app/services/live_market_data.py and app/services/district_market_data.py for the two data.gov.in resources, with app/services/mandi_directory.py performing market-name discovery/matching in front of both, as described in Sections 10.8 and 13.4.

16.5 Authentication and Localisation Implementation

Authentication is implemented with python-jose (JWT) and bcrypt password hashing, plus a separate server-side-only admin credential path. Localisation is implemented as fifteen language JSON translation files, backend NLU/intent/template modules, and a live-served capability matrix (Section 10.19), rather than a call to an external translation or LLM API.

16.6 Deployment

The live frontend is deployed at cropwise-alpha.vercel.app (Vercel). Backend deployment guidance in the project's documentation targets Render, with an explicit operational warning: because the database is a single SQLite file on local disk (DATABASE_URL=sqlite:///./cropwise.db by default), most Render web-service plans treat local disk as ephemeral, wiping it on every redeploy or restart unless a Render Disk (persistent volume) is attached and DATABASE_URL is pointed at a path inside it (for example sqlite:////var/data/cropwise.db, noting the required four slashes for an absolute path). This is documented as a property of the SQLite-plus-ephemeral-filesystem combination generally, not a defect introduced by any specific change, and the project's documentation recommends backing up the database file before any deploy to a service holding real user data if disk persistence has not been confirmed.

16.7 External Data Sources and Fallback/Seeded Data

By default, and with zero external API keys or internet dependency required, CropWise runs entirely on realistic seeded demonstration data: ten real Chhattisgarh markets, ten crops, sixty days of synthesised historical mandi prices generated by a deterministic seeded random walk (so the demo is stable and reproducible), and demo farmer/buyer accounts. When DATA_GOV_IN_API_KEY is set and MARKET_DATA_SOURCE=live is configured, live market-price lookups are attempted first via the two data.gov.in resources described in Section 13.4, falling back transparently to the seeded dataset if neither live source has a record for that combination or both are unreachable. Every price record is individually labelled data_source: "live" or "demo" — never blended — and historical charts/forecasts use only genuine accumulated live data by default, with an explicit include_demo=true opt-in (itself tagged is_demo: true in the response) to view the synthetic 60-day series for methodology walkthroughs.

The project's own documentation includes an important honesty note on the limits of what has actually been verified: because api.data.gov.in was outside the network egress allowlist available while this feature was built, a live successful fetch from either government resource has not been directly observed by the system's own developers/AI-assisted build process — only the fallback path (non-200 response, timeout, or no records) has been exercised, which is exactly the failure mode the fallback logic is designed for. The discovery, matching, merge, and persistence logic compiles and is structured to be testable with mocked responses, but has not yet been watched resolving a real Chhattisgarh mandi or district against the live government dataset. This paper reports that limitation as-is rather than presenting the live-data path as fully verified.

17. Screenshots

This paper does not include fabricated or simulated screenshots of the CropWise application. Genuine screenshots of the running application (landing page, farmer dashboard, marketplace, price intelligence, arrival intelligence, lot management, buyer dashboard, buyer demands, AI advisor, group selling, transactions, admin dashboard, and the multilingual interface) should be captured directly from the live deployment at https://cropwise-alpha.vercel.app/ and inserted into this section, each with a figure number, caption, and short explanatory note, before this paper is finalised for submission. A placeholder screenshots/ folder has been created in the accompanying project package (Section 26) for this purpose; capturing and inserting the actual images is listed as an outstanding action item in the delivery summary that accompanies this paper.

18. Results and Discussion

This section documents genuine, verifiable results from the current implementation, as recorded in the project's own repository documentation. No adoption, revenue, market-impact, or ML-performance figures are reported, because no such formal measurement has been conducted or documented for this system.

18.1 Functional Modules Delivered

All twenty functional module areas described in Section 10 have working frontend pages and corresponding backend routers/services, running end-to-end on seeded demonstration data with zero required configuration. Sixteen backend routers and roughly fifteen frontend pages are organised one-to-one with the feature areas described in this paper's introduction and module sections.

18.2 Automated Testing Status

The project's own documentation records 18 of 18 automated tests passing for the login-tracking feature specifically (backend/tests/test_login_tracking.py, covering unique-user counting, failed-login identity handling, and admin-only access control). The same documentation states plainly that no broader Python test suite currently exists in the repository beyond this file, and that a full npm run build and a real successful data.gov.in fetch were not verified during the most recent documented development pass because the build sandbox used at that time had no network access. This paper reports these facts as-is rather than inferring broader test coverage than what is documented.

18.3 Security Fixes Verified in Documentation

The project's changelog documents a specific, itemised set of security fixes as applied and verified in the current build: admin endpoints now require the require_admin role guard (previously /admin/impact was publicly reachable); the JWT secret is never a hard-coded, publicly visible value (a random one is generated at startup with a logged warning if SECRET_KEY is unset); public farmer/buyer endpoints no longer leak email/phone (a PII-free public schema is returned, with contact details only via authenticated /me endpoints); marketplace offers are validated server-side as described in Section 13.3; group-selling pool membership no longer double-counts on re-join; and the buyer-matching endpoint now requires the listing owner's farmer login (previously unauthenticated). The exposed data.gov.in API key referenced in the same changelog has been documented as rotated, and a project .gitignore has been added where none previously existed.

18.4 Frontend–Backend and Database Integration

All touched Python files in the most recent documented development pass pass python -m py_compile, indicating the backend is at minimum syntactically sound and importable. The documented lightweight, additive-only migration routine (Section 14.2) is designed specifically so that shipping updates to a live Render deployment with real user data does not require a destructive schema change, an operational property that is itself a result worth recording rather than assuming.

18.5 Discussion

Taken together, these results indicate a functionally complete demonstration system covering the full breadth of the PS-26132 problem statement, with an honest and explicit internal accounting of which parts are backed by real government data (or capable of being, pending live-network verification) versus seeded demonstration data, and which decision-support components are rule-based/statistical versus genuinely machine-learned. The system's most significant unverified area is the live government-data integration path itself (Section 16.7): the logic is implemented and unit-testable with mocked responses, but has not been observed resolving a real request against the live data.gov.in endpoints due to sandbox network restrictions during development. This is discussed further as a limitation in Section 21 rather than presented as a resolved capability.

19. Testing

Testing evidence in this paper is limited strictly to what the project's own repository documents; no additional test executions were performed by this paper's authors beyond what is already recorded in the project.

Test Case	Expected Result	Actual Result (as documented)	Status
Login tracking — unique-user counting	Repeated logins by the same user counted once as a unique login	Verified by backend/tests/test_login_tracking.py	PASS
Login tracking — failed-login identity handling	A failed login attempt is recorded without leaking whether the account exists or storing the raw email	Verified by backend/tests/test_login_tracking.py	PASS
Login tracking — admin-only access	GET /admin/user-activity, /admin/recent-activity, /admin/users reject non-admin callers	Verified by backend/tests/test_login_tracking.py	PASS
Python syntax/compile check	All touched backend files compile without syntax errors	python -m py_compile passes on all touched files	PASS
Live data.gov.in successful fetch	A live request to either government resource returns real records	Not observed — api.data.gov.in was outside the build sandbox's network egress allowlist; only the fallback (non-200/timeout/no-records) path was exercised	NOT VERIFIED
Frontend production build (npm run build)	The Vite production build completes without errors	Not verified in the most recent documented pass (no network in the build sandbox)	NOT VERIFIED
Broader backend automated test suite	Comprehensive endpoint/service-level automated tests across all sixteen routers	No such suite currently exists beyond the login-tracking tests	NOT PRESENT

No claim of manual UI/UX testing, load testing, or penetration testing is made in this paper, because no such testing is documented in the project's own materials. Rows marked NOT VERIFIED or NOT PRESENT are reported honestly rather than omitted, consistent with this paper's accuracy requirements.

20. Advantages

- Integrated market information: price, arrival, and net-profit calculation are presented together rather than as raw, disconnected data.
- Direct farmer-buyer connectivity through a verified, server-side-validated marketplace rather than reliance on intermediaries alone.
- Improved price visibility through a transparent, government-data-capable price pipeline with honest live/demo labelling.
- Digital lot (harvest listing) management covering the full lifecycle from posting to accepted offer.
- Explainable decision support: every AgriAdvisor recommendation and buyer-match ranking shows its contributing factors rather than functioning as a black box.

- Cooperative tools (FarmPool, Group Selling) that can reduce per-unit transport cost for smallholders who cannot individually amortise logistics expense.
- Multilingual accessibility architecture that keeps language strictly a presentation-layer concern, making it comparatively low-cost to extend language coverage without touching business logic.
- Modular service-oriented backend architecture that avoids duplicated business logic across features sharing the same underlying computation (for example transport cost).
- Administrative visibility via a live-computed impact dashboard and privacy-conscious login-activity tracking.
- Zero-dependency demonstrability: the full system runs end-to-end with no external API keys or internet dependency via seeded data, while retaining a real integration path to live government data.

21. Limitations

- Limited real-world validation: the platform has not been piloted with real farmers or buyers; all documented usage is against seeded demonstration data or, at most, unit-tested logic.
- Live government-data integration is implemented but unverified end-to-end: a live successful fetch from either data.gov.in resource has not been directly observed due to network restrictions in the development sandbox; only the fallback path has been exercised.
- A verified filter-field bug exists in the live market-data fetch path (filters[state] versus the correct state.keyword per the live OpenAPI spec), and district/market/commodity/variety/grade filters require the same audit.
- Transport-cost modelling uses an unrealistically simple flat rate (₹7.5/km/tonne plus a ₹60–150 base fee) with no diesel-price variable, vehicle-class differentiation, or minimum-trip economics — a limitation the project's own specification explicitly flags.
- No trained machine-learning model is used anywhere in the system; AgriAdvisor, the price forecaster, the buyer matcher, and quality grading are rule-based or statistical, which limits their predictive sophistication relative to a data-driven model, in exchange for explainability.
- Quality assessment is a deterministic simulation keyed off the crop name or filename, not an actual computer-vision analysis of an uploaded image; no real image upload/storage pipeline exists yet.
- Weather-risk input to the recommendation engine is a deterministic simulated signal, not a live weather-API integration.
- Voice interaction (speech-to-text) is reliable only in English and Hindi by design; other supported UI languages lack a working microphone-based interaction.
- Freeform marketplace text (offer messages, listing notes) is not machine-translated across languages; original text and its source language are preserved and shown as-is rather than translated, because no translation provider is currently configured.
- Only ten Chhattisgarh markets and ten crops are covered by the seeded dataset; broader geographic and crop coverage requires additional data-entry work.
- SQLite is used for zero-configuration demonstration; a production deployment with concurrent real users would likely require migrating to PostgreSQL, and the current Render deployment guidance requires careful attention to disk persistence to avoid data loss on redeploy.

- No formal usability testing has been conducted with the platform's actual target user base (smallholder farmers with varying literacy and digital-access levels), despite the platform's accessibility-oriented design goals.
- No comprehensive backend automated test suite exists beyond the login-tracking tests; broader regression coverage across the sixteen routers is not yet in place.
- A full audit of buyer-verification enforcement, hard-coded-string i18n coverage, and responsive-layout breakpoints has not been completed, per the project's own stated scope for a subsequent development pass.

22. Future Scope

The following items are proposed future work. None of them are implemented in the current system, and none should be read as a claim about present functionality.

- Verify and harden the live data.gov.in integration with real network access, correcting the known filter-field bug and completing the district/market/commodity/variety/grade filter audit.
- Introduce a trained price-forecasting model (for example Prophet, XGBoost, or an LSTM network) behind the existing price_predictor.py interface, once sufficient genuine historical data has accumulated from live usage.
- Replace the deterministic quality-grading simulation with a trained computer-vision model behind the existing service boundary, paired with a real image-upload/storage pipeline.
- Integrate a live weather-data API in place of the current simulated weather-risk signal within the recommendation engine.
- Introduce more realistic transport-cost modelling incorporating current diesel price, vehicle class, and minimum-trip economics.
- Extend geographic and crop coverage beyond the current ten Chhattisgarh markets and ten crops used in the seeded dataset.
- Advance the recommendation and buyer-matching logic from rule-based scoring toward a hybrid model that retains explainability while incorporating learned components, as more real usage data becomes available.
- Broaden voice-first interaction beyond English and Hindi as browser/OS speech-recognition quality for additional Indian languages improves, or as a dedicated third-party speech provider is integrated.
- Integrate a real machine-translation provider (implementing the existing TranslationProvider interface) for freeform marketplace text across languages.
- Conduct structured usability studies and, subject to appropriate ethical and institutional approval, a real-world pilot deployment with farmer and buyer cooperatives.
- Expand automated test coverage across all sixteen backend routers and the frontend, beyond the current login-tracking-focused suite.
- Migrate the production database from SQLite to PostgreSQL and adopt a persistent-disk or managed-database deployment strategy to remove the current ephemeral-storage risk on platforms such as Render.

- Integrate satellite or IoT-derived signals (for example remote-sensing crop-condition indices) as additional inputs to the recommendation engine, subject to data availability and cost.
- Complete the outstanding internal audits noted in Section 21 (buyer-verification enforcement, hard-coded-string i18n coverage, responsive-layout breakpoints, UI-level data-provenance badge audit) as part of ongoing quality assurance.

23. Security and Privacy

This section documents security and privacy mechanisms actually present in the current implementation, based on the project's own documentation; no control is claimed here that is not documented as implemented.

23.1 Authentication and Authorization

Farmer and buyer accounts authenticate via JWT (python-jose) with bcrypt-hashed passwords. Admin access uses a separate, server-side-only credential path (ADMIN_USERNAME/ADMIN_PASSWORD environment variables) that is never hard-coded, never present in the frontend bundle or source, and is checked exclusively in app/routers/auth.py::admin_login — the admin login form on the frontend only ever forwards whatever the user types to that endpoint. Admin-only endpoints (the Impact Dashboard and User Activity endpoints) are protected by a require_admin role guard; this replaced a prior state in which /admin/impact was publicly reachable, a fix documented in the project's own changelog.

23.2 Input Validation

All request bodies and query parameters are validated against Pydantic schemas. Marketplace offer submission specifically enforces positive price/quantity, an offer quantity not exceeding the listing's quantity, rejection of offers below the farmer's minimum acceptable price, and prevention of duplicate accept/reject actions against an inactive listing (Section 13.3).

23.3 Secret Management

The JWT signing secret is never a publicly visible hard-coded value: if SECRET_KEY is not set in the environment, the backend generates a random one at startup and logs a warning, rather than falling back to a known placeholder value. The previously exposed DATA_GOV_IN_API_KEY has been documented as rotated, and a project .gitignore has been added to prevent .env files from being committed in future, where none previously existed.

23.4 Personally Identifiable Information (PII) Handling

Public farmer/buyer endpoints return a PII-free schema; email and phone number are included only in the authenticated /me endpoint responses for the account holder themself. The login_events table is deliberately designed to never store passwords, password hashes, tokens, API keys, IP addresses, or the raw email typed during a login attempt — only a numeric user id (when the attempt matched a real account), role, timestamp, and success/failure are recorded.

23.5 Role-Based Access Control

The buyer-matching endpoint requires the authenticated listing-owner farmer's own login (a documented fix — it was previously unauthenticated). Group-selling pool membership updates are scoped to the authenticated farmer's own membership row, preventing one farmer's action from corrupting another's pool-quantity data.

23.6 Data Protection Caveats

This paper does not claim encryption-at-rest, formal penetration testing, rate limiting, or GDPR/DPDP-specific compliance mechanisms, because none of these are documented as implemented in the project's current materials. These are appropriately treated as future hardening work (Section 22) rather than existing controls.

24. Conclusion

This paper has presented CropWise, a full-stack digital platform addressing SIH problem statement PS-26132, "Strengthening Market Linkages and Price Discovery for Farmers." The core contribution is architectural and integrative: CropWise brings market price intelligence, transparent net-profit calculation, an explainable rule-based selling recommendation, a verified farmer-to-buyer marketplace, multi-factor buyer matching, shared-transport and group-selling tools, and multilingual text/voice access together under one connected system, in a market where these capabilities currently exist, at best, in separate and disconnected government or private-platform silos.

The system's implementation choices reflect a consistent design philosophy of honesty over apparent sophistication: decision-support logic is implemented as explainable rules and transparent statistics rather than as opaque machine-learning models; every price and forecast record is explicitly labelled as live government data or seeded demonstration data rather than silently blended; and known implementation gaps — an incorrect live-data filter field, an unrealistic flat-rate transport model, an unverified live-fetch path, and a simulated rather than trained quality-grading model — are documented candidly in the project's own materials and reproduced candidly in this paper, rather than smoothed over.

The current system is a functionally complete demonstration covering the full breadth of the problem statement's scope, running end-to-end on realistic seeded data with zero external dependency, while retaining a genuine, testable (though not yet live-verified) integration path to government open data. Its principal limitations are the absence of real-world pilot validation, an unverified live government-data path, simplified transport-cost and quality-assessment models, and the absence of trained machine-learning components anywhere in the decision-support pipeline. Future work — detailed in Section 22 — centres on verifying and hardening the live-data integration, introducing trained forecasting and vision models behind existing, already-designed swap points, broadening geographic and language coverage, and conducting structured usability and pilot studies with real farmer and buyer users. Taken together, CropWise demonstrates a credible, technically grounded, and honestly scoped approach to closing the market-linkage and price-discovery gap identified in PS-26132, while leaving a clear and realistic path toward production-grade deployment.

25. References

The following references support the background, motivation, and literature review sections of this paper. Only sources that are genuinely verifiable government, official, or well-established reference sources are listed; no fabricated papers, authors, journals, or DOIs are included.

10. Government of India, Ministry of Agriculture & Farmers Welfare. National Agriculture Market (eNAM). https://enam.gov.in

11. Government of India, Directorate of Marketing and Inspection. AGMARKNET — Agricultural Marketing Information Network. https://agmarknet.gov.in

12. Government of India, Open Government Data (OGD) Platform. data.gov.in — "Current Daily Price of Various Commodities from Various Markets" (Resource ID: 9ef84268-d588-465a-a308-a864a43d0070). https://data.gov.in

13. Government of India, Open Government Data (OGD) Platform. data.gov.in — "Variety-wise Daily Market Prices Data of Commodity" (Resource ID: 35985678-0d79-46b4-9ed6-6f13308a1d24). https://data.gov.in

14. Smart India Hackathon 2026, Problem Statement PS-26132: "Strengthening Market Linkages and Price Discovery for Farmers," Government of Maharashtra (Maharashtra State Innovation Society). https://sih.gov.in

15. FastAPI Official Documentation. https://fastapi.tiangolo.com

16. React Official Documentation. https://react.dev

17. SQLAlchemy Official Documentation. https://www.sqlalchemy.org

18. Vite Official Documentation. https://vitejs.dev

19. Mozilla Developer Network. Web Speech API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

20. Render Documentation. Render Disks (Persistent Storage). https://render.com/docs/disks

21. Abha153/cropwise GitHub Repository (project source and documentation referenced throughout this paper). https://github.com/Abha153/cropwise

22. CropWise Live Deployment. https://cropwise-alpha.vercel.app/

Note: several of the adoption-barrier and market-linkage findings summarised in Section 4 (for example eNAM adoption studies across Uttar Pradesh, Punjab, and Tamil Nadu, and industry estimates on agritech value-pool composition) are drawn from the project's own prior internally-conducted research synthesis ("CropWise — Mode 1 Research," PS-26132 research note) rather than from newly located peer-reviewed sources for this paper; readers seeking the specific underlying studies should consult recent eNAM-adoption and Indian-agritech-market literature directly, as specific author/journal citations were not independently re-verified during the preparation of this paper.

26. Appendix

A. Detailed Router / API Category Table

Router / Feature Area	Representative Endpoints (illustrative)	Auth Required
Authentication	POST /auth/login, POST /auth/register, POST /auth/admin/login	No (issues token) / Yes for admin
Farmer	GET /farmer/me, GET /farmer/{id} (public, PII-free)	Mixed
Buyer	GET /buyer/me, GET /buyer/{id}	Mixed

Router / Feature Area	Representative Endpoints (illustrative)	Auth Required
	(public, PII-free)	
Marketplace (AgriMarket)	POST /listings, GET /listings, POST /listings/{id}/offers	Yes
Buyer Verification	GET/POST /buyer/verification	Yes
Buyer Demands	GET/POST /buyer/demands	Yes
Buyer Matching	GET /listings/{id}/matches	Yes (listing owner only)
Market / Price Intelligence	GET /market/prices, GET /market/compare, GET /market/live-markets, GET /market/data-source-status	Mixed
Forecast	GET /forecast	Mixed
Assistant	POST /assistant/query, GET /assistant/languages	Mixed
FarmPool / Group Pooling	GET/POST /pool	Yes
Quality Grading	POST /quality/grade	Yes
Alerts	GET /alerts	Yes
Admin	GET /admin/impact, GET /admin/user-activity, GET /admin/recent-activity, GET /admin/users	Yes (admin only)

Endpoint paths above are representative and organised by documented feature area; exact route strings should be confirmed against the live OpenAPI documentation at /docs on a running backend instance, since this paper does not reproduce the full source tree verbatim.

B. Supported Languages Summary

Fifteen languages are fully supported (UI translated, assistant intent/entity extraction, native-language responses): English, Hindi, Marathi, Bengali, Tamil, Telugu, Gujarati, Kannada, Malayalam, Punjabi, Odia, Assamese, Urdu (RTL), Bhojpuri, and Maithili — with voice input/output verified reliable only for English and Hindi, and device-dependent (attempted, gracefully falling back to text on error) for the remaining thirteen. A further twenty-three languages (including Sanskrit, Nepali, Konkani, Kashmiri, Sindhi, Manipuri, Bodo, Dogri, Santali, and several international languages such as Mandarin, Japanese, Korean, Spanish, French, German, Portuguese, Arabic, Russian, Indonesian, Vietnamese, Thai, Turkish, and Italian) are selectable in the UI with an honest English-fallback badge for AI understanding and native response.

C. Demo Accounts (Non-Sensitive, Publicly Documented)

Role	Account	Notes
Farmer	ramesh@cropwise.demo	Bilaspur · Tomato, Paddy, Soybean
Farmer	sunita@cropwise.demo	Raigarh · Onion, Wheat, Maize
Farmer	manoj@cropwise.demo	Durg · Maize, Chana, Groundnut
Buyer	freshfoods@cropwise.demo	Processor, Raipur
Buyer	greenbasket@cropwise.demo	Retailer, Bilaspur
Buyer	agriexport@cropwise.demo	Exporter, Durg

(Demo password for all seeded accounts, per the project's public README: demo1234. These are non-production, seeded demonstration credentials, publicly documented by the project itself, and are reproduced here for completeness rather than as a security disclosure.)

D. Project Directory Structure (as documented)

cropwise/backend/app/{main.py, config.py, database.py, models.py, schemas.py, auth_utils.py, seed_data.py, i18n/, mock_data/, services/, routers/}; cropwise/frontend/src/{api/, context/, i18n/, components/, pages/}. See the project README for the complete, current tree.

E. Technical Configuration Notes

Environment variables and their working defaults are documented in backend/.env.example and frontend/.env.example within the source repository; both provide working defaults for local development and require editing only when changing ports or deploying beyond localhost. No secret values are reproduced in this paper.